

Advanced C Programming Lab

Session VIII – May 14

Objectives

The topic we will be covering;

- C Structures (structs)
- C typedef

Structures

Structures (also called structs) are a way to group several related variables into one place. Each variable in the structure is known as a member of the structure.

Unlike an array, a structure can contain many different data types (`int, float, char`, etc.).

To create a structure, we use the `struct` keyword and declare each of its members inside curly braces:

```
struct myInfo {
// Structure declaration
  int myAge;
// Member (int variable)
  char myCls;
// Member (char variable)
};
// End the structure with a
semicolon
```

Structures

```c
#include <stdio.h>
// Create a structure called myInfo
struct myInfo {
  int myAge;
  char myCls;
};
int main() {
  // Create a structure variable of
myInfo called i1
  struct myInfo i1;
  // Assign values to members of s1
  i1.myAge = 22;
  i1.myCls = 'B';
  // Print values
  printf("My age is: %d\n", i1.myAge);
  printf("My class is: %c\n", i1.myCls);
  return 0;
}
```

In order to access the structure, we need to create a variable of it using the `struct` keyword inside of our `main()` code.

Output:

My age is: 22
My class is: B

Structures

```
#include <stdio.h>
#include <string.h>
// A structure called myInfo
struct myInfo {
  int myAge;
  char myCls;
  char myInit[10]; // String
};
int main() {
  struct myInfo i1;
   // Assign a value to the string using the
strcpy function
  strcpy(i1.myInit, "RDJ");
  // Print value
  printf("My initials are: %s\n", i1.myInit);
  return 0;
}
```

But what if we want to use strings in structures? As discussed before, strings are basically an array of characters, so we need to assign some empty space for them to fill. Also, we need to use `strcpy()` function to assign the desired value.

Output:

My initials are: RDJ

Structures

```
#include <stdio.h>
#include <string.h>
// A structure called myInfo
struct myInfo {
  int myAge;
  char myCls;
  char myInit[10]; // String
};
int main() {
   // Create a structure variable and assign
values to it
  struct myInfo i1 = {24, 'A', "SR"};
   // Print value
  printf("%d %c %s", i1.myAge, i1.myCls,
i1.myInit);
  return 0;
}
```

By using curly braces {} , we can make the code much simpler;

Output:

24 A SR

Structures

We can also copy and assign one structure to another, while still being able to modify the values in each one if we need to do so.

```
struct myStructure s1 = {13, 'B', "Some text"};
struct myStructure s2;
// Copy s1 values to s2
s2 = s1;
```

Structures

```c
#include <stdio.h>
#include <string.h>
struct Car {
  char brand[10];
  char model[10];
  int year;
};
int main() {
  struct Car car1  = {"BMW", "740i", 1999};
  struct Car car2;
  car2 = car1;
  strcpy(car2.model, "750iL");
  car2.year = 1998;
  printf("%s %s %d\n", car1.brand, car1.model, car1.year);
  printf("%s %s %d\n", car2.brand, car2.model, car2.year);
return 0;
}
```

Output:

BMW 740i 1999
BMW 750iL 1998

Structs and Pointers

We can use pointers with structs to make our code more efficient. One instance is when passing structs to functions or changing their values.

To use a pointer to a struct, just add the * symbol, like you would with other data types.
To access its members, you must use the -> operator instead of the dot . syntax.

Pointer are helpful especially when we want to avoid copying large amounts of data or changing values inside a function.

Structs and Pointers

```c
#include <stdio.h>
// Define a struct
struct Car {
  char brand[10];
  int year;
};
int main() {
  struct Car car = {"Peugeot", 2005};
  // Declare a pointer to the struct
  struct Car *ptr = &car;
  // Access members using the → operator
  printf("Brand: %s\n", ptr→brand);
  printf("Year: %d\n", ptr→year);
  return 0;
}
```

Output:

Brand: Peugeot
Year: 2005

typedef

The typedef keyword lets you create a new name (an alias) for an existing type. This can make complex declarations easier to read, and your code easier to maintain.

For example, instead of always writing `float`, we can create a new type called `Temperature` to make the code clearer.

typedef

Here, the typedef creates an alias Temp for the int type. This allows us to use Temp instead of int throughout the program, making the code potentially more readable.

```
#include <stdio.h>
typedef float Temp;
int main() {
  Temp today = 31.5;
  Temp tomorrow = 27.6;
  printf("Today: %.1f C\n", today);
  printf("Tomorrow: %.1f C\n", tomorrow);
  return 0;
}
```

Output:

```
Today: 31.5 C
Tomorrow: 27.6 C
```

typedef

```c
#include <stdio.h>
struct Car {  // Without typedef
  char brand[10];
  int year;
};
typedef struct { // With typedef
  char brand[10];
  int year;
} Car;
int main() {
  struct Car car1 = {"Porsche", 1995}; // needs "struct"
  Car car2 = {"DS", 1969}; // shorter with typedef
  printf("%s %d\n", car1.brand, car1.year);
  printf("%s %d\n", car2.brand, car2.year);
  return 0;
}
```

In modern C, `typedef` is often used together with `struct`, `enum`, and function pointers to keep code clean and easier to read.

Output:

```
Porsche 1995
DS 1969
```

Thank you for your time and attention!

References:

- https://www.w3schools.com/
- https://www.geeksforgeeks.org/